The Journal of Open Source Software

plyxp: Efficient and Tidy Manipulation of Annotated Matrix Data in R/Bioconductor

Justin T. Landis [1,2¶] **and Michael I. Love** [2,3]

1 Bioinformatics and Computational Biology Program, University of North Carolina at Chapel Hill, United States **2** Department of Genetics, University of North Carolina at Chapel Hill, United States **3** Department of Biostatistics, University of North Carolina at Chapel Hill, United States ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 23 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Many biological experiments produce matrices containing measurements of features by samples. These measurements are only interpretable alongside two additional tables of descriptive information: one describing the features and one describing the samples. All three pieces must stay in sync whenever the data are filtered, reordered, or transformed; otherwise a measurement can silently become attached to the wrong feature or sample. *plyxp* is an R package that lets analysts manipulate these annotated matrix data using the user-friendly verbs of the *dplyr* package (Wickham et al., 2023), such as `mutate()` and `filter()`, while guaranteeing that measurements and descriptions remain aligned. It introduces a small, explicit grammar for stating which of the three contexts an expression refers to and where its result should be stored. *plyxp* is distributed through Bioconductor and can be installed with `BiocManager::install("plyxp")`; the source code is available at https://github.com/jtlandis/plyxp.

Statement of need

High-throughput omics data are commonly summarized as one or more matrices of counts or abundances across genomic features and samples or cells. Analyses routinely require computations that link the matrix to its metadata, such as dividing RNA-seq counts by transcript length or by library size. Data classes such as *ExpressionSet* (Gentleman et al., 2004) and *SummarizedExperiment* (SE) (Huber et al., 2015) in R/Bioconductor, and *AnnData* (Virshup et al., 2024) in Python, keep data and metadata consistent during subsetting, reordering, and binding.

Operating on these objects, however, requires learning domain-specific accessor functions. Grouped computations over sets of rows or columns often require explicit loops or less familiar functional programming helpers, which can obscure the purpose of analysis code. The *tidyomics* project (Hutchison et al., 2024) addresses this by bringing the chained, verb-based style of tidy data analysis (Wickham, 2014), as implemented in *dplyr* and the wider *tidyverse* (Wickham et al., 2019), to Bioconductor classes.

plyxp targets Bioconductor users whose analysis pipelines require SE objects for their compact structure but who want to adopt tidy-style abstractions that do not sacrifice efficiency. Instead of converting the object to a long table as a naive use of *dplyr* would require, *plyxp* evaluates expressions directly against the matrix and metadata in place, using the data-masking facilities of *rlang* (Henry & Wickham, 2026), and returns an object of the same class that was supplied. *plyxp* establishes flexible access across matrices and metadata in that the user chooses extracting data "as-is" or in a convenient "reshaped" form (see code example in Software design section).

DRAFT

The Journal of Open Source Software

State of the field

Within *tidyomics*, tidy interfaces exist for genomic ranges (*plyranges*) (Lee et al., 2019, 2020), count matrices (*tidybulk*) (Mangiola, Molania, et al., 2021), and Seurat objects (*tidyseurat*) (Mangiola, Doyle, et al., 2021). The closest existing tool is *tidySummarizedExperiment*, released with Bioconductor 3.12 in 2020, which presents an SE as an abstracted data frame by conceptually unwinding the matrix into a tidy long-form table. This design is well suited to exploration and to piping data directly into *ggplot2*, but computations that combine matrix values with metadata pay the cost of that unwinding and of reconstructing the SE afterwards. The long-form view also discards information about which table a new variable belongs to and requires rules for resolving name collisions across tables. In Python, *AnnData* provides the data structure, and *pandas* (McKinney, 2022) provides chained table operations, but no comparable grammar connects the two to our knowledge.

These limitations follow from the long-form abstraction itself. We therefore built *plyxp* as a separate package with its own class, *PlySummarizedExperiment*, which wraps any SE subclass (e.g., *SingleCellExperiment*) by composition. Because the class is distinct, *plyxp* and *tidySummarizedExperiment* can be loaded together without method conflicts, and *tidySummarizedExperiment* now calls *plyxp* when possible (see Research impact statement).

Software design

plyxp defines three evaluation contexts for an object with n rows and p columns: `assays`, one or more $n \times p$ matrices; `rows`, vectors of length n describing features; and `cols`, vectors of length p describing samples. Expressions are evaluated in the `assays` context by default, and the syntax functions `rows()` and `cols()` redirect evaluation to the metadata (Figure 1A). The same syntax determines where results are stored by `mutate()` and which dimension is subset by `filter()`. Below we create a new *plyxp* object (`xp`) from a SE where the rows and columns of the matrices represent genes and samples, respectively (setup code in the companion repository (Landis & Love, 2026)):

```
xp <- new_plyxp(se)
xp |>
  mutate(
    log_counts = log10(counts + 1),
    cols(treated = dex == "trt"),
    cpbp = counts / .rows$gene_length,
    cpm = counts / .cols$mil_of_reads
  ) |>
  filter(rows(gene_biotype == "lincRNA"))
```

DRAFT

The Journal of Open Source Software

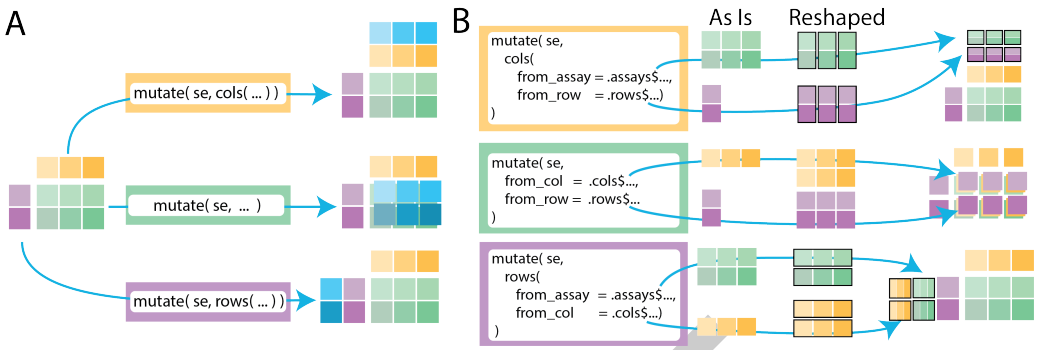


Figure 1: Diagram of the *plyxp* syntax with the *dplyr* `mutate()` function, where the `assays`, `rows`, and `cols` contexts are represented by the green, purple, and yellow blocks, respectively. (A) Syntax within the `mutate()` call determines the context in which new values (blue blocks) are inserted. (B) Pronouns allow references between contexts. Pronouns first access the requested data "as-is" before reshaping it for the current context. Black bounding boxes indicate elements of a list.

Contexts are connected through *rlang* pronouns, objects that give an unambiguous lookup into another environment (Wickham, 2019). Each context is reachable through two pronouns (Figure 1B). The reshaped pronoun (e.g., `.rows`, `.assays`) returns data resized to fit the current context, so the row-metadata vector `gene_length` above is replicated to the size of the matrix. This enables convenient and efficient per-element arithmetic to generate new assays, such as counts-per-basepair (`cpbp`) and counts-per-million (`cpm`) above. The as-is pronoun (e.g., `.assays_asis`) returns the data exactly as stored, so that optimized routines can operate on its native shape. For example, both expressions below add row means to the row metadata, but the first uses the vectorized `rowMeans()` on the matrix, whereas the second receives the matrix reshaped as a list of rows and iterates over it with *purrr* (Wickham & Henry, 2025):

```
xp |>
  mutate(rows(
    ave  = rowMeans(.assays_asis$counts),
    ave2 = purrr::map_dbl(.assays$counts, mean)
  ))
```

Internally, each context is an *rlang* data mask, a chain of environments in which object data and pronoun targets are bound lazily. Thus, a cross-context transformation is computed only if the user's expression requests it, and at most once per data mask (Figure 2). This avoids eagerly converting the whole object, eliminates name collisions between contexts, and makes the destination of every result unambiguous. Grouping is likewise constrained by the matrix structure. *plyxp* only permits groupings that partition whole rows or columns, because arbitrary partitions of matrix elements are not guaranteed to form valid sub-matrices. Enforcing this restriction for grouping has enabled *plyxp* to implement the *dplyr* `summarise()` function such that it will return an SE object. Prior implementations of `summarise()` for SE objects returned a data frame because the dropped structural information and unrestricted grouping posed a challenge to consistently constructing an SE object.

DRAFT

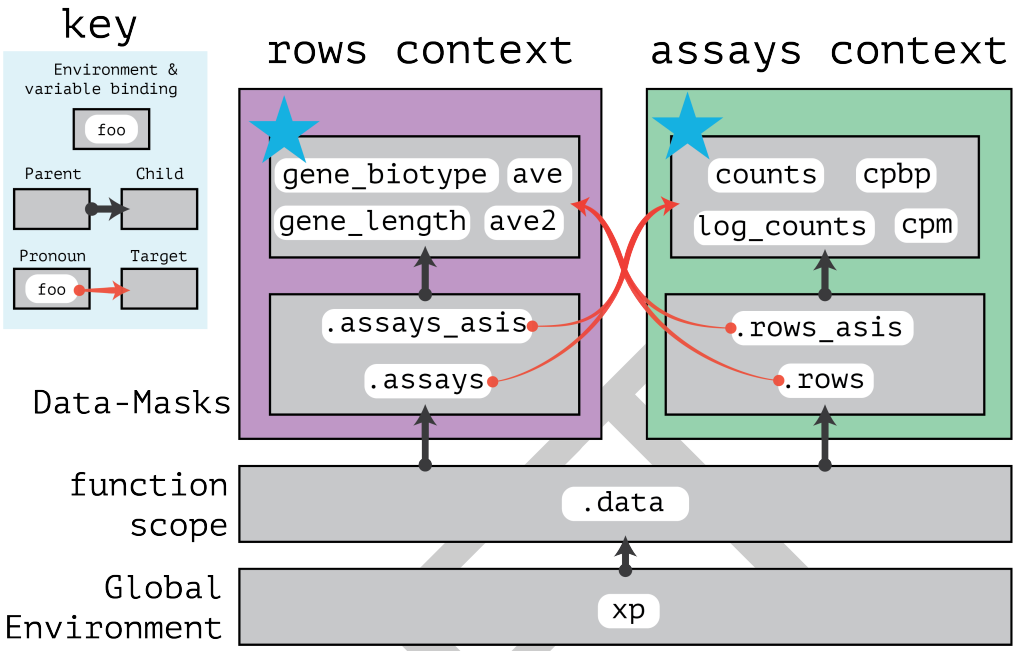


Figure 2: Simplified data-mask structure in *plyxp*. The `assays` and `rows` contexts are represented by the green and purple blocks, respectively. Grey rectangles represent environments and contain example variable bindings. Parent environments are linked to child environments by black lines, and pronoun bindings are linked to their target contexts by red lines. For brevity, neither full environment hierarchies nor all extended environments are shown. Entry points for each context are indicated by blue stars. Through *rlang*'s tidy evaluation engine, symbols within the data mask are resolved before the calling environment is searched.

plyxp builds its abstractions upon Bioconductor-native operations such as `[` subsetting and incurs a small, fixed overhead. Although *plyxp* is not expected to outperform direct calls to the SE interface, its `group_by()`/`summarise()` operations are more expressive and often faster than equivalent hand-written code (Landis & Love, 2026).

Research impact statement

plyxp is a realized example of a new expressive grammar for access and manipulation of annotated matrix data and a framework for efficiently performing the operations implied by the syntax. Its three evaluation contexts combined with the proposed sentinel syntax provide an unambiguous grammar that is faithful to the structure of the annotated matrix. Specifically, each evaluation context is distinct such that objects within are of a similar "size" (e.g., for an annotated matrix of $n \times p$ dimensions, the rows, columns, and assays are size n, p and $n \times p$, respectively). Furthermore, identical symbols between these contexts will represent different objects, and the sentinel functions `rows()` and `cols()` provide a clear indication of which context is being referenced. Cross-context operations are optimized in that any reshaping is done lazily and may be avoided entirely through the "as-is" pronoun variants, exemplifying the flexibility of *plyxp*.

We benchmarked a representative cross-context task, scaling the counts matrix by gene length, on the *airway* RNA-seq dataset (Himes et al., 2014). *plyxp* was more than an order of magnitude faster than unwinding the object to a long table and back, and within an order of magnitude of hand-written native R code. Within *plyxp*, choosing the as-is pronoun over the reshaped one reduced the time needed to compute row means roughly tenfold, showing why both access modes are exposed. Benchmark code and the package versions used are available

DRAFT

in a companion repository (Landis & Love, 2026).

plyxp has also been adopted outside our group. As of the Bioconductor 3.22 release (October 2025), *tidySummarizedExperiment* uses *plyxp* internally in place of operations that previously required a long-form representation, so its users benefit without changing their code. *plyxp* is distributed through both the release and development branches of Bioconductor, where it passes the project's automated build and check process, and it was downloaded from more than 600 distinct IP addresses in the first month of the Bioconductor 3.23 release (May 2026).

AI usage disclosure

Generative AI tools were not used in the development of the *plyxp* software, its documentation (vignettes and reference manual), or the authoring of the original manuscript from which this paper is derived. An AI assistant (Claude Opus 5.5, Anthropic) was used to restructure and condense that existing manuscript text to meet the Journal of Open Source Software's format and length requirements, and to draft the repository's contributing guide. All resulting text was reviewed, edited, and verified by the human authors, who made all design, analysis, and interpretation decisions.

Acknowledgements

The authors thank Rachel Sharp, Beatriz Campillo Miñano, Stuart Lee, and Stefano Mangiola for comments on an earlier draft. This work is supported in part by an Essential Open Source Software (EOSS) grant from the Wellcome Trust [313919/Z/24/Z] and by the National Institutes of Health grant R01-HG009937. The funders had no role in the design or implementation of *plyxp*. The authors declare no competing interests.

References

Gentleman, R. C., Carey, V. J., Bates, D. M., Bolstad, B., Dettling, M., Dudoit, S., Ellis, B., Gautier, L., Ge, Y., Gentry, J., Hornik, K., Hothorn, T., Huber, W., Iacus, S., Irizarry, R., Leisch, F., Li, C., Maechler, M., Rossini, A. J., … Zhang, J. (2004). Bioconductor: Open software development for computational biology and bioinformatics. *Genome Biology*, *5*(10), R80.

Henry, L., & Wickham, H. (2026). *Rlang: Functions for base types and core r and 'tidyverse' features*. https://doi.org/10.32614/CRAN.package.rlang

Himes, B. E., Jiang, X., Wagner, P., Hu, R., Wang, Q., Klanderman, B., Whitaker, R. M., Duan, Q., Lasky-Su, J., Nikolos, C., Jester, W., Johnson, M., Panettieri, R. A., Jr, Tantisira, K. G., Weiss, S. T., & Lu, Q. (2014). RNA-Seq transcriptome profiling identifies CRISPLD2 as a glucocorticoid responsive gene that modulates cytokine function in airway smooth muscle cells. *PLOS One*, *9*(6), e99625.

Huber, W., Carey, V. J., Gentleman, R., Anders, S., Carlson, M., Carvalho, B. S., Bravo, H. C., Davis, S., Gatto, L., Girke, T., Gottardo, R., Hahne, F., Hansen, K. D., Irizarry, R. A., Lawrence, M., Love, M. I., MacDonald, J., Obenchain, V., Oleś, A. K., … Morgan, M. (2015). Orchestrating high-throughput genomic analysis with bioconductor. *Nature Methods*, *12*(2), 115–121.

Hutchison, W. J., Keyes, T. J., tidyomics Consortium, Crowell, H. L., Serizay, J., Soneson, C., Davis, E. S., Sato, N., Moses, L., Tarlinton, B., Nahid, A. A., Kosmac, M., Clayssen, Q., Yuan, V., Mu, W., Park, J.-E., Mamede, I., Ryu, M. H., Axisa, P.-P., … Mangiola, S. (2024). The tidyomics ecosystem: Enhancing omic data analyses. *Nature Methods*, *21*(7), 1166–1170.

DRAFT

Landis, J. T., & Love, M. I. (2026). *Jtlandis/plyxp_paper: v1.2.1* (Version v1.2.1). Zenodo. https://doi.org/10.5281/zenodo.23601311

Lee, S., Cook, D., & Lawrence, M. (2019). Plyranges: A grammar of genomic data transformation. *Genome Biology*, *20*(1), 4. https://doi.org/10.1186/s13059-018-1597-8

Lee, S., Lawrence, M., & Love, M. I. (2020). Fluent genomics with plyranges and tximeta. *F1000Research*, *9*, 109.

Mangiola, S., Doyle, M. A., & Papenfuss, A. T. (2021). Interfacing seurat with the R tidy universe. *Bioinformatics*, *37*(22), 4100–4107.

Mangiola, S., Molania, R., Dong, R., Doyle, M. A., & Papenfuss, A. T. (2021). Tidybulk: An R tidy framework for modular transcriptomic data analysis. *Genome Biology*, *22*(1), 42.

McKinney, W. (2022). *Python for data analysis 3e* (3rd ed.). O'Reilly Media.

Virshup, I., Rybakov, S., Theis, F. J., Angerer, P., & Wolf, F. A. (2024). Anndata: Access and store annotated data matrices. *Journal of Open Source Software*, *9*(101), 4371. https://doi.org/10.21105/joss.04371

Wickham, H. (2014). Tidy data. *Journal of Statistical Software*, *59*(10), 1–23. https://doi.org/10.18637/jss.v059.i10

Wickham, H. (2019). *Advanced r, second edition* (2nd ed.). Chapman & Hall/CRC.

Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., … Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, *4*(43), 1686. https://doi.org/10.21105/joss.01686

Wickham, H., François, R., Henry, L., Müller, K., & Vaughan, D. (2023). *Dplyr: A grammar of data manipulation*. https://doi.org/10.32614/CRAN.package.dplyr

Wickham, H., & Henry, L. (2025). *Purrr: Functional programming tools*. https://doi.org/10.32614/CRAN.package.purrr

DRAFT